

DJANGI TOOLBOX

// SOURCE CODE EXPORT • 20 file(s) • 2026-10-03 23:51:43

App.axaml

F:\Tools\DjangoToolbox\App.axaml

```
1 | <Application xmlns="https://github.com/avaloniaui"
2 |     xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
3 |     x:Class="DjangoToolbox.App"
4 |     xmlns:local="using:DjangoToolbox"
5 |     RequestedThemeVariant="Default">
6 |     <!-- "Default" ThemeVariant follows system theme variant. "Dark" or "Light" are other available options. -->
7 |
8 |     <Application.DataTemplates>
9 |         <local:ViewLocator />
10 |    </Application.DataTemplates>
11 |
12 |    <Application.Styles>
13 |        <FluentTheme />
14 |        <StyleInclude Source="avares://DjangoToolbox/Themes/Cyberpunk.axaml"/>
15 |    </Application.Styles>
16 |
17 |    <Application.Resources>
18 |        <FontFamily x:Key="AppMonoFont">avares://DjangoToolbox/Assets/Fonts#JetBrains Mono</FontFamily>
19 |    </Application.Resources>
20 | </Application>
```

App.axaml.cs

F:\Tools\DjangoToolbox\App.axaml.cs

```
1 | using System;
2 | using System.IO;
3 | using System.Reflection;
4 | using Avalonia;
5 | using Avalonia.Controls.ApplicationLifetimes;
6 | using Avalonia.Markup.Xaml;
7 | using DjangoToolbox.ViewModels;
8 | using DjangoToolbox.Views;
9 | using QuestPDF.Drawing;
10 | using QuestPDF.Infrastructure;
11 |
12 | namespace DjangoToolbox;
13 |
14 | public partial class App : Application
15 | {
16 |     public override void Initialize()
17 |     {
18 |         QuestPDF.Settings.License = LicenseType.Community;
19 |         RegisterQuestPdfFonts();
20 |         AvaloniaXamlLoader.Load(this);
21 |     }
22 |
23 |     private void RegisterQuestPdfFonts()
24 |     {
25 |         // Get the current assembly where the fonts are embedded
26 |         var assembly = Assembly.GetExecutingAssembly();
27 |
28 |         // List the font files you want to register.
29 |         // The resource name format is: [AssemblyName].Assets.Fonts.[FileName]
30 |         // For example: "Djangitoolbox.Assets.Fonts.JetBrainsMono-Regular.ttf"
31 |         var fontResources = new[]
32 |         {
33 |             "Djangitoolbox.Assets.Fonts.JetBrainsMono-Regular.ttf", // Replace with your actual font file name
34 |             "Djangitoolbox.Assets.Fonts.Lato-Regular.ttf"           // Replace with your actual font file name
35 |         };
36 |
37 |         foreach (var resourceName in fontResources)
38 |         {
39 |             using var fontStream = assembly.GetManifestResourceStream(resourceName);
40 |             if (fontStream != null)
41 |             {
42 |                 // Register the font stream with QuestPDF.
43 |                 // The family name (e.g., "JetBrains Mono") is read from the font file itself.
44 |                 FontManager.RegisterFontFromStream(fontStream);
45 |             }
46 |             else
47 |             {
48 |                 // Log or handle the case where the font resource is missing
49 |                 Console.WriteLine($"Warning: Could not find embedded font resource '{resourceName}'");
50 |             }
51 |         }
52 |     }
53 |
54 |     public override void OnFrameworkInitializationCompleted()
55 |     {
56 |         if (ApplicationLifetime is IClassicDesktopStyleApplicationLifetime desktop)
57 |         {
58 |             desktop.MainWindow = new MainWindow
59 |             {
60 |                 DataContext = new MainWindowViewModel(),
61 |             };
62 |         }
63 |     }
64 | }
```

DJANGI TOOLBOX

// SOURCE CODE EXPORT • 20 file(s) • 2026-10-03 23:51:43

```
62 |         }
63 |
64 |         base.OnFrameworkInitializationCompleted();
65 |     }
66 | }
```

Program.cs

F:\Tools\DjangoToolbox\Program.cs

```
1 | using Avalonia;
2 | using System;
3 |
4 | namespace DjangoToolbox;
5 |
6 | sealed class Program
7 | {
8 |     // Initialization code. Don't use any Avalonia, third-party APIs or any
9 |     // SynchronizationContext-reliant code before AppMain is called: things aren't initialized
10 |    // yet and stuff might break.
11 |    [STAThread]
12 |    public static void Main(string[] args)
13 |    {
14 |        BuildAvaloniaApp().StartWithClassicDesktopLifetime(args);
15 |    }
16 |
17 |    // Avalonia configuration, don't remove; also used by visual designer.
18 |    public static AppBuilder BuildAvaloniaApp()
19 |        => AppBuilder.Configure<App>()
20 |            .UsePlatformDetect()
21 |            #if DEBUG
22 |                .WithDeveloperTools()
23 |            #endif
24 |                .WithInterFont()
25 |                .LogToTrace();
26 | }
```

ViewLocator.cs

F:\Tools\DjangoToolbox\ViewLocator.cs

```
1 | using System;
2 | using System.Diagnostics.CodeAnalysis;
3 | using Avalonia.Controls;
4 | using Avalonia.Controls.Templates;
5 | using DjangoToolbox.ViewModels;
6 |
7 | namespace DjangoToolbox;
8 |
9 | /// <summary>
10 | /// Given a view model, returns the corresponding view if possible.
11 | /// </summary>
12 | [RequiresUnreferencedCode(
13 |     "Default implementation of ViewLocator involves reflection which may be trimmed away.",
14 |     Url = "https://docs.avaloniaui.net/docs/concepts/view-locator")]
15 | public class ViewLocator : IDataTemplate
16 | {
17 |     public Control? Build(object? param)
18 |     {
19 |         if (param is null)
20 |             return null;
21 |
22 |         var name = param.GetType().FullName!.Replace("ViewModel", "View", StringComparison.Ordinal);
23 |         var type = Type.GetType(name);
24 |
25 |         if (type != null)
26 |         {
27 |             return (Control)Activator.CreateInstance(type)!;
28 |         }
29 |
30 |         return new TextBlock { Text = "Not Found: " + name };
31 |     }
32 |
33 |     public bool Match(object? data)
34 |     {
35 |         return data is ViewModelBase;
36 |     }
37 | }
```

FileEntry.cs

F:\Tools\DjangoToolbox\Models\FileEntry.cs

```
1 | using System;
2 | using System.IO;
3 |
4 | namespace DjangoToolbox.Models;
5 |
6 | public class FileEntry
7 | {
8 |     public string Path { get; }
9 |     public string Name { get; }
```

DJANGI TOOLBOX

// SOURCE CODE EXPORT • 20 file(s) • 2026-10-03 23:51:43

```
10 |     public string SizeDisplay { get; }
11 |
12 |     public FileEntry(string path)
13 |     {
14 |         Path = path;
15 |         Name = System.IO.Path.GetFileName(path);
16 |         try
17 |         {
18 |             var bytes = new FileInfo(path).Length;
19 |             SizeDisplay = FormatSize(bytes);
20 |         }
21 |         catch
22 |         {
23 |             SizeDisplay = "?";
24 |         }
25 |     }
26 |
27 |     private static string FormatSize(long bytes) => bytes switch
28 |     {
29 |         < 1024 => $"{bytes} B",
30 |         < 1024 * 1024 => $"{bytes / 1024.0:F1} KB",
31 |         _ => $"{bytes / (1024.0 * 1024.0):F1} MB"
32 |     };
33 | }
```

LogEntry.cs

F:\Tools\DjangoToolbox\Models\LogEntry.cs

```
1 | using System;
2 |
3 | namespace DjangoToolbox.Models;
4 |
5 | public class LogEntry
6 | {
7 |     public string TimeText { get; }
8 |     public string Level { get; }
9 |     public string Message { get; }
10 |
11 |     // Boolean flags for XAML class bindings
12 |     public bool IsInfo => Level == "INFO";
13 |     public bool IsWarn => Level == "WARN";
14 |     public bool IsError => Level == "ERROR";
15 |
16 |     public LogEntry(string message, string level = "INFO")
17 |     {
18 |         TimeText = DateTime.Now.ToString("HH:mm:ss");
19 |         Level = level;
20 |         Message = message;
21 |     }
22 | }
```

FilePicker.cs

F:\Tools\DjangoToolbox\Services\FilePicker.cs

```
1 | using Avalonia.Controls;
2 |
3 | namespace DjangoToolbox.Services;
4 |
5 | /// <summary>
6 | /// Global access to the app's TopLevel, so ViewModels can open
7 | /// native file/folder pickers without holding a Window reference.
8 | /// </summary>
9 | public static class FilePicker
10 | {
11 |     public static TopLevel? Instance { get; private set; }
12 |
13 |     public static void Register(TopLevel topLevel) => Instance = topLevel;
14 | }
```

PdfGenerator.cs

F:\Tools\DjangoToolbox\Services\PdfGenerator.cs

```
1 | using System;
2 | using System.Collections.Generic;
3 | using System.IO;
4 | using System.Text;
5 | using DjangoToolbox.Models;
6 | using QuestPDF.Fluent;
7 | using QuestPDF.Helpers;
8 | using QuestPDF.Infrastructure;
9 |
10 | namespace DjangoToolbox.Services;
11 |
12 | public static class PdfGenerator
13 | {
14 |     public static void Generate(string outputPath, IReadOnlyList<FileEntry> files)
15 |     {
16 |         Document.Create(container =>
17 |         {
```

DJANGI TOOLBOX

// SOURCE CODE EXPORT • 20 file(s) • 2026-10-03 23:51:43

```
18 |         container.Page(page =>
19 |         {
20 |             page.Size(PageSizes.A4);
21 |             page.Margin(24);
22 |             page.DefaultTextStyle(t => t
23 |                 .FontFamily("JetBrains Mono")
24 |                 .FontSize(7)
25 |                 .LineHeight(1.2f));
26 |
27 |             // ---- Header (repeats on every page) ----
28 |             page.Header().PaddingBottom(6).Column(h =>
29 |             {
30 |                 h.Spacing(2);
31 |                 h.Item().Text("DJANGI TOOLBOX")
32 |                     .FontSize(14).Bold().FontColor("#1F2A36");
33 |                 h.Item().Text(
34 |                     $"// SOURCE CODE EXPORT ? {files.Count} file(s) ? {DateTime.Now:yyyy-MM-dd HH:mm:ss}")
35 |                     .FontSize(7).FontColor("#8A99A8");
36 |                 h.Item().PaddingTop(2).LineHorizontal(0.5f).LineColor("#1F2A36");
37 |             });
38 |
39 |             // ---- Content: all files, continuous ----
40 |             page.Content().Column(col =>
41 |             {
42 |                 col.Spacing(0);
43 |
44 |                 foreach (var file in files)
45 |                 {
46 |                     // File header block
47 |                     col.Item().PaddingTop(8).PaddingBottom(2).Column(fh =>
48 |                     {
49 |                         fh.Spacing(1);
50 |                         fh.Item().Text(file.Name)
51 |                             .FontSize(9).Bold().FontColor("#1F2A36");
52 |                         fh.Item().Text(file.Path)
53 |                             .FontSize(6).FontColor("#8A99A8");
54 |                         fh.Item().PaddingTop(1)
55 |                             .LineHorizontal(0.5f).LineColor("#1F2A36");
56 |                     });
57 |
58 |                     string[] lines;
59 |                     try
60 |                     {
61 |                         lines = File.ReadAllLines(file.Path);
62 |                     }
63 |                     catch
64 |                     {
65 |                         col.Item().Text("// unable to read file")
66 |                             .FontColor("#FF003C");
67 |                         continue;
68 |                     }
69 |
70 |                     // Code lines ? no truncation
71 |                     int lineNo = 0;
72 |                     foreach (var raw in lines)
73 |                     {
74 |                         lineNo++;
75 |                         var escaped = Escape(raw);
76 |                         col.Item().Text(text =>
77 |                         {
78 |                             text.Span(lineNo.ToString().PadLeft(5) + " | ")
79 |                                 .FontColor("#8A99A8");
80 |                             text.Span(escaped)
81 |                                 .FontColor("#0A0E14");
82 |                         });
83 |                     }
84 |                 }
85 |             });
86 |
87 |             // ---- Footer ----
88 |             page.Footer().PaddingTop(4).Row(row =>
89 |             {
90 |                 row.RelativeItem().Text("DJANGI TOOLBOX // codezipper")
91 |                     .FontSize(6).FontColor("#8A99A8");
92 |
93 |                 row.ConstantItem(100).AlignRight().Text(text =>
94 |                 {
95 |                     text.DefaultTextStyle(t => t.FontSize(6).FontColor("#8A99A8"));
96 |                     text.Span("Page ");
97 |                     text.CurrentPageNumber();
98 |                     text.Span(" / ");
99 |                     text.TotalPages();
100 |                 });
101 |             });
102 |         });
103 |     })
104 |     .GeneratePdf(outputPath);
105 | }
```

DJANGI TOOLBOX

// SOURCE CODE EXPORT • 20 file(s) • 2026-10-03 23:51:43

```
106 |
107 | private static string Escape(string line)
108 | {
109 |     var sb = new StringBuilder(line.Length);
110 |     foreach (var c in line)
111 |     {
112 |         if (c == '\t') sb.Append("    ");
113 |         else if (c < 32) sb.Append(' ');
114 |         else if (c > 126) sb.Append('?');
115 |         else sb.Append(c);
116 |     }
117 |     return sb.ToString();
118 | }
119 | }
```

Cyberpunk.axaml

F:\Tools\DjangiToolbox\Themes\Cyberpunk.axaml

```
1 | <Styles xmlns="https://github.com/avaloniaui"
2 |     xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml">
3 |
4 |     <Styles.Resources>
5 |         <ResourceDictionary>
6 |             <!-- ===== Cyberpunk 2077 palette ===== -->
7 |             <SolidColorBrush x:Key="CyberYellow" Color="#FCE22A"/>
8 |             <SolidColorBrush x:Key="CyberCyan" Color="#5EF2FA"/>
9 |             <SolidColorBrush x:Key="CyberRed" Color="#FF003C"/>
10 |            <SolidColorBrush x:Key="CyberGreen" Color="#3DFFA0"/>
11 |
12 |            <SolidColorBrush x:Key="CyberBg" Color="#0A0E14"/>
13 |            <SolidColorBrush x:Key="CyberPanel" Color="#10151C"/>
14 |            <SolidColorBrush x:Key="CyberBorder" Color="#1F2A36"/>
15 |            <SolidColorBrush x:Key="CyberDim" Color="#8A99A8"/>
16 |
17 |            <FontFamily x:Key="CyberFont">aavars://DjangiToolbox/Assets/Fonts#JetBrains Mono</FontFamily>
18 |        </ResourceDictionary>
19 |    </Styles.Resources>
20 |
21 |    <!-- =====
22 |    <!-- BUTTONS
23 |    <!-- =====
24 |    <Style Selector="Button.icon">
25 |        <Setter Property="Padding" Value="0"/>
26 |        <Setter Property="HorizontalContentAlignment" Value="Center"/>
27 |        <Setter Property="VerticalContentAlignment" Value="Center"/>
28 |        <Setter Property="FontSize" Value="14"/>
29 |    </Style>
30 |
31 |    <Style Selector="Button">
32 |        <Setter Property="Foreground" Value="{DynamicResource CyberYellow}"/>
33 |        <Setter Property="Padding" Value="22,10"/>
34 |        <Setter Property="FontFamily" Value="{DynamicResource CyberFont}"/>
35 |        <Setter Property="FontWeight" Value="SemiBold"/>
36 |        <Setter Property="FontSize" Value="14"/>
37 |        <Setter Property="Cursor" Value="Hand"/>
38 |        <Setter Property="Transitions">
39 |            <Transitions>
40 |                <BrushTransition Property="Foreground" Duration="0:0:0.15"/>
41 |            </Transitions>
42 |        </Setter>
43 |    </Style>
44 |
45 |    <!-- Restyle the actual rendered rectangle of the button -->
46 |    <Style Selector="Button /template/ ContentPresenter#PART_ContentPresenter">
47 |        <Setter Property="Background" Value="#0F141B"/>
48 |        <Setter Property="BorderBrush" Value="{DynamicResource CyberYellow}"/>
49 |        <Setter Property="BorderThickness" Value="1"/>
50 |        <Setter Property="Transitions">
51 |            <Transitions>
52 |                <BrushTransition Property="Background" Duration="0:0:0.15"/>
53 |                <BrushTransition Property="BorderBrush" Duration="0:0:0.15"/>
54 |            </Transitions>
55 |        </Setter>
56 |    </Style>
57 |
58 |    <Style Selector="Button:pointerover /template/ ContentPresenter#PART_ContentPresenter">
59 |        <Setter Property="Background" Value="{DynamicResource CyberYellow}"/>
60 |        <Setter Property="BorderBrush" Value="{DynamicResource CyberYellow}"/>
61 |        <Setter Property="Foreground" Value="#0A0E14"/>
62 |    </Style>
63 |
64 |    <Style Selector="Button:pointerover">
65 |        <Setter Property="Foreground" Value="#0A0E14"/>
66 |        <Setter Property="Effect">
67 |            <DropShadowEffect Color="#FCE22A" BlurRadius="18" OffsetX="0" OffsetY="0" Opacity="0.75"/>
68 |        </Setter>
69 |    </Style>
70 |
```

DJANGI TOOLBOX

// SOURCE CODE EXPORT • 20 file(s) • 2026-10-03 23:51:43

```
71 | <Style Selector="Button:pressed /template/ ContentPresenter#PART_ContentPresenter">
72 |     <Setter Property="Background" Value="{DynamicResource CyberCyan}" />
73 | </Style>
74 |
75 | <Style Selector="Button:pressed">
76 |     <Setter Property="Foreground" Value="#0A0E14" />
77 |     <Setter Property="Effect">
78 |         <DropShadowEffect Color="#5EF2FA" BlurRadius="22" OffsetX="0" OffsetY="0" Opacity="0.9" />
79 |     </Setter>
80 | </Style>
81 |
82 | <Style Selector="Button.close">
83 |     <Setter Property="Foreground" Value="{DynamicResource CyberRed}" />
84 | </Style>
85 |
86 | <Style Selector="Button.close:pointerover /template/ ContentPresenter#PART_ContentPresenter">
87 |     <Setter Property="Background" Value="{DynamicResource CyberRed}" />
88 |     <Setter Property="BorderBrush" Value="{DynamicResource CyberRed}" />
89 |     <Setter Property="Foreground" Value="White" />
90 | </Style>
91 |
92 | <Style Selector="Button.close:pointerover">
93 |     <Setter Property="Foreground" Value="White" />
94 |     <Setter Property="Effect">
95 |         <DropShadowEffect Color="#FF003C" BlurRadius="18" OffsetX="0" OffsetY="0" Opacity="0.85" />
96 |     </Setter>
97 | </Style>
98 |
99 | <Style Selector="Button.active /template/ ContentPresenter#PART_ContentPresenter">
100 |     <Setter Property="Background" Value="{DynamicResource CyberYellow}" />
101 |     <Setter Property="BorderBrush" Value="{DynamicResource CyberYellow}" />
102 |     <Setter Property="Foreground" Value="#0A0E14" />
103 | </Style>
104 |
105 | <Style Selector="Button.active">
106 |     <Setter Property="Foreground" Value="#0A0E14" />
107 | </Style>
108 |
109 | <!-- =====>
110 | <!-- TEXT -->
111 | <!-- =====>
112 | <Style Selector="TextBlock">
113 |     <Setter Property="FontFamily" Value="{DynamicResource CyberFont}" />
114 |     <Setter Property="Foreground" Value="{DynamicResource CyberCyan}" />
115 | </Style>
116 |
117 | <Style Selector="TextBlock.hero">
118 |     <Setter Property="FontSize" Value="42" />
119 |     <Setter Property="FontWeight" Value="Bold" />
120 |     <Setter Property="Foreground" Value="{DynamicResource CyberYellow}" />
121 | </Style>
122 |
123 | <Style Selector="TextBlock.dim">
124 |     <Setter Property="FontSize" Value="12" />
125 |     <Setter Property="Foreground" Value="{DynamicResource CyberDim}" />
126 | </Style>
127 |
128 | <!-- =====>
129 | <!-- PANELS (bordered containers) -->
130 | <!-- =====>
131 | <Style Selector="Border.panel">
132 |     <Setter Property="Background" Value="{DynamicResource CyberPanel}" />
133 |     <Setter Property="BorderBrush" Value="{DynamicResource CyberBorder}" />
134 |     <Setter Property="BorderThickness" Value="1" />
135 |     <Setter Property="Padding" Value="16" />
136 | </Style>
137 |
138 | <Style Selector="Border.accent">
139 |     <Setter Property="BorderBrush" Value="{DynamicResource CyberYellow}" />
140 | </Style>
141 |
142 | <!-- Scrollbars: thin, cyan -->
143 | <Style Selector="ScrollBar">
144 |     <Setter Property="Background" Value="Transparent" />
145 | </Style>
146 | <Style Selector="ScrollBar /template/ Thumb">
147 |     <Setter Property="Background" Value="#1F2A36" />
148 | </Style>
149 | <Style Selector="ScrollBar:pointerover /template/ Thumb">
150 |     <Setter Property="Background" Value="{DynamicResource CyberCyan}" />
151 | </Style>
152 |
153 | <!-- Log level colors -->
154 | <Style Selector="TextBlock.info">
155 |     <Setter Property="Foreground" Value="{DynamicResource CyberCyan}" />
156 | </Style>
157 | <Style Selector="TextBlock.warn">
158 |     <Setter Property="Foreground" Value="{DynamicResource CyberYellow}" />
```

DJANGI TOOLBOX

// SOURCE CODE EXPORT • 20 file(s) • 2026-10-03 23:51:43

```
159 |     </Style>
160 |     <Style Selector="TextBlock.error">
161 |         <Setter Property="Foreground" Value="{DynamicResource CyberRed}" />
162 |     </Style>
163 |
164 |
165 |     <!-- =====
166 |     GLITCH BUTTON ? layered ghosts + RGB split on hover
167 |     =====>
168 |     <Style Selector="Button.glitch">
169 |         <Setter Property="Template">
170 |             <ControlTemplate>
171 |                 <Grid>
172 |                     <!-- Cyan ghost -->
173 |                     <ContentPresenter Name="GhostCyan"
174 |                         Content="{TemplateBinding Content}"
175 |                         Foreground="{DynamicResource CyberCyan}"
176 |                         HorizontalContentAlignment="{TemplateBinding HorizontalContentAlignment}"
177 |                         VerticalContentAlignment="{TemplateBinding VerticalContentAlignment}"
178 |                         Padding="{TemplateBinding Padding}"
179 |                         IsHitTestVisible="False"
180 |                         Opacity="0"
181 |                         ZIndex="0" />
182 |
183 |                     <!-- Red ghost -->
184 |                     <ContentPresenter Name="GhostRed"
185 |                         Content="{TemplateBinding Content}"
186 |                         Foreground="{DynamicResource CyberRed}"
187 |                         HorizontalContentAlignment="{TemplateBinding HorizontalContentAlignment}"
188 |                         VerticalContentAlignment="{TemplateBinding VerticalContentAlignment}"
189 |                         Padding="{TemplateBinding Padding}"
190 |                         IsHitTestVisible="False"
191 |                         Opacity="0"
192 |                         ZIndex="1" />
193 |
194 |                     <!-- Real button surface -->
195 |                     <ContentPresenter Name="PART_ContentPresenter"
196 |                         Content="{TemplateBinding Content}"
197 |                         Background="{TemplateBinding Background}"
198 |                         BorderBrush="{TemplateBinding BorderBrush}"
199 |                         BorderThickness="{TemplateBinding BorderThickness}"
200 |                         CornerRadius="{TemplateBinding CornerRadius}"
201 |                         Foreground="{TemplateBinding Foreground}"
202 |                         HorizontalContentAlignment="{TemplateBinding HorizontalContentAlignment}"
203 |                         VerticalContentAlignment="{TemplateBinding VerticalContentAlignment}"
204 |                         Padding="{TemplateBinding Padding}"
205 |                         ZIndex="2" />
206 |                 </Grid>
207 |             </ControlTemplate>
208 |         </Setter>
209 |     </Style>
210 |
211 |     <!-- Initial ghost state -->
212 |     <Style Selector="Button.glitch /template/ ContentPresenter#GhostCyan">
213 |         <Setter Property="Opacity" Value="0" />
214 |         <Setter Property="TranslateTransform.X" Value="0" />
215 |     </Style>
216 |     <Style Selector="Button.glitch /template/ ContentPresenter#GhostRed">
217 |         <Setter Property="Opacity" Value="0" />
218 |         <Setter Property="TranslateTransform.X" Value="0" />
219 |     </Style>
220 |
221 |     <!-- Cyan ghost animation on hover -->
222 |     <Style Selector="Button.glitch:pointerover /template/ ContentPresenter#GhostCyan">
223 |         <Style.Animations>
224 |             <Animation Duration="0:0:0.35" IterationCount="1">
225 |                 <KeyFrame Cue="0%">
226 |                     <Setter Property="Opacity" Value="0.9" />
227 |                     <Setter Property="TranslateTransform.X" Value="-4" />
228 |                 </KeyFrame>
229 |                 <KeyFrame Cue="18%">
230 |                     <Setter Property="TranslateTransform.X" Value="3" />
231 |                 </KeyFrame>
232 |                 <KeyFrame Cue="34%">
233 |                     <Setter Property="TranslateTransform.X" Value="-2" />
234 |                 </KeyFrame>
235 |                 <KeyFrame Cue="52%">
236 |                     <Setter Property="TranslateTransform.X" Value="2" />
237 |                 </KeyFrame>
238 |                 <KeyFrame Cue="75%">
239 |                     <Setter Property="Opacity" Value="0.35" />
240 |                     <Setter Property="TranslateTransform.X" Value="-1" />
241 |                 </KeyFrame>
242 |                 <KeyFrame Cue="100%">
243 |                     <Setter Property="Opacity" Value="0" />
244 |                     <Setter Property="TranslateTransform.X" Value="0" />
245 |                 </KeyFrame>
246 |             </Animation>
```

DJANGI TOOLBOX

// SOURCE CODE EXPORT • 20 file(s) • 2026-10-03 23:51:43

```
247 |         </Style.Animations>
248 |     </Style>
249 |
250 |     <!-- Red ghost animation on hover (offset timing ? slice feel) -->
251 |     <Style Selector="Button.glitch:pointerover /template/ ContentPresenter#GhostRed">
252 |         <Style.Animations>
253 |             <Animation Duration="0:0:0.35" IterationCount="1" Delay="0:0:0.04">
254 |                 <KeyFrame Cue="0%">
255 |                     <Setter Property="Opacity" Value="0.9"/>
256 |                     <Setter Property="TranslateTransform.X" Value="4"/>
257 |                 </KeyFrame>
258 |                 <KeyFrame Cue="15%">
259 |                     <Setter Property="TranslateTransform.X" Value="-3"/>
260 |                 </KeyFrame>
261 |                 <KeyFrame Cue="38%">
262 |                     <Setter Property="TranslateTransform.X" Value="2"/>
263 |                 </KeyFrame>
264 |                 <KeyFrame Cue="60%">
265 |                     <Setter Property="Opacity" Value="0.4"/>
266 |                     <Setter Property="TranslateTransform.X" Value="-1"/>
267 |                 </KeyFrame>
268 |                 <KeyFrame Cue="100%">
269 |                     <Setter Property="Opacity" Value="0"/>
270 |                     <Setter Property="TranslateTransform.X" Value="0"/>
271 |                 </KeyFrame>
272 |             </Animation>
273 |         </Style.Animations>
274 |     </Style>
275 |
276 |     <!-- PULSE -->
277 |     <Style Selector="Border.logRow">
278 |         <Setter Property="TranslateTransform.X" Value="0"/>
279 |         <Style.Animations>
280 |             <Animation Duration="0:0:0.55" IterationCount="1">
281 |                 <KeyFrame Cue="0%">
282 |                     <Setter Property="Opacity" Value="0"/>
283 |                     <Setter Property="TranslateTransform.X" Value="-12"/>
284 |                 </KeyFrame>
285 |                 <KeyFrame Cue="35%">
286 |                     <Setter Property="Opacity" Value="1"/>
287 |                     <Setter Property="TranslateTransform.X" Value="0"/>
288 |                 </KeyFrame>
289 |                 <KeyFrame Cue="55%">
290 |                     <Setter Property="Opacity" Value="0.55"/>
291 |                 </KeyFrame>
292 |                 <KeyFrame Cue="100%">
293 |                     <Setter Property="Opacity" Value="1"/>
294 |                     <Setter Property="TranslateTransform.X" Value="0"/>
295 |                 </KeyFrame>
296 |             </Animation>
297 |         </Style.Animations>
298 |     </Style>
299 |
300 | </Styles>
```

BootViewModel.cs

F:\Tools\DjangiToolbox\ViewModels\BootViewModel.cs

```
1 | using System.Collections.ObjectModel;
2 | using System.Threading.Tasks;
3 | using CommunityToolkit.Mvvm.ComponentModel;
4 |
5 | namespace DjangiToolbox.ViewModels;
6 |
7 | public partial class BootViewModel : ViewModelBase
8 | {
9 |     // The scrolling log lines
10 |    public ObservableCollection<string> Lines { get; } = new();
11 |
12 |    [ObservableProperty]
13 |    // [NotifyPropertyChangedFor(nameof(ProgressWidth))]
14 |    [NotifyPropertyChangedFor(nameof(ProgressPercent))]
15 |    private double _progress;
16 |
17 |    [ObservableProperty]
18 |    private bool _isComplete;
19 |
20 |    [ObservableProperty]
21 |    private bool _isVisible = true;
22 |
23 |    // Progress bar is 600px wide in XAML ? keep these in sync if you change it.
24 |    // public double ProgressWidth => Progress * 600;
25 |    public string ProgressPercent => $"{(int)(Progress * 100),3}%";
26 |
27 |    public async Task RunAsync()
28 |    {
29 |        await Task.Delay(250);
30 |    }
```


DJANGI TOOLBOX

// SOURCE CODE EXPORT • 20 file(s) • 2026-10-03 23:51:43

```
31 |         await Line("> INITIALIZING BOOT SEQUENCE ..... [OK]", 160);
32 |         await Line("> LOADING KERNEL MODULES ..... [OK]", 180);
33 |         await Line("> MOUNTING VIRTUAL FILESYSTEM ..... [OK]", 150);
34 |         await Line("> ESTABLISHING NEURAL LINK ..... [OK]", 220);
35 |         await Line("> DECRYPTING OPERATOR CREDENTIALS ..... [OK]", 200);
36 |         await Line("> LOADING DJANGI TOOLBOX CORE ..... [OK]", 220);
37 |
38 |         // Progress bar fill
39 |         for (int i = 0; i ≤ 100; i++)
40 |         {
41 |             Progress = i / 100.0;
42 |             await Task.Delay(10);
43 |         }
44 |
45 |         await Line("", 80);
46 |         await Line("> ACCESS GRANTED", 350);
47 |         IsComplete = true;
48 |     }
49 |
50 |     private async Task Line(string text, int delayMs)
51 |     {
52 |         Lines.Add(text);
53 |         await Task.Delay(delayMs);
54 |     }
55 | }
```

MainWindowViewModel.cs

F:\Tools\DjangiToolbox\ViewModels\MainWindowViewModel.cs

```
1 | using System;
2 | using System.Threading.Tasks;
3 | using CommunityToolkit.Mvvm.ComponentModel;
4 | using CommunityToolkit.Mvvm.Input;
5 | using DjangiToolbox.ViewModels.Pages;
6 |
7 | namespace DjangiToolbox.ViewModels;
8 |
9 | public partial class MainWindowViewModel : ViewModelBase
10 | {
11 |     public BootViewModel Boot { get; } = new();
12 |
13 |     private readonly HomeViewModel _home = new();
14 |     private readonly CodeZipperViewModel _codeZipper = new();
15 |
16 |     public event Func<Task>? BeforePageChange;
17 |     public event Func<Task>? AfterPageChange;
18 |
19 |     [ObservableProperty]
20 |     private ViewModelBase _currentPage;
21 |
22 |     [ObservableProperty]
23 |     [NotifyPropertyChangedFor(nameof(IsHomeActive))]
24 |     [NotifyPropertyChangedFor(nameof(IsZipActive))]
25 |     private string _activeKey = "home";
26 |
27 |     public bool IsHomeActive ⇒ ActiveKey == "home";
28 |     public bool IsZipActive ⇒ ActiveKey == "zip";
29 |
30 |     public MainWindowViewModel()
31 |     {
32 |         _currentPage = _home;
33 |     }
34 |
35 |     [RelayCommand]
36 |     private Task NavigateHome() ⇒ SetPageAsync("home", _home);
37 |
38 |     [RelayCommand]
39 |     private Task NavigateCodeZipper() ⇒ SetPageAsync("zip", _codeZipper);
40 |
41 |     private async Task SetPageAsync(string key, ViewModelBase page)
42 |     {
43 |         if (ActiveKey == key) return;
44 |
45 |         if (BeforePageChange is not null)
46 |             await BeforePageChange.Invoke();
47 |
48 |         ActiveKey = key;
49 |         CurrentPage = page;
50 |
51 |         if (AfterPageChange is not null)
52 |             await AfterPageChange.Invoke();
53 |     }
54 | }
```

ViewModelBase.cs

F:\Tools\DjangiToolbox\ViewModels\ViewModelBase.cs

```
1 | using CommunityToolkit.Mvvm.ComponentModel;
2 |
```

DJANGI TOOLBOX

// SOURCE CODE EXPORT • 20 file(s) • 2026-10-03 23:51:43

```
3 | namespace DjangoToolbox.ViewModels;
4 |
5 | public abstract class ViewModelBase : ObservableObject
6 | {
7 | }
```

MainWindow.axaml

F:\Tools\DjangoToolbox\Views\MainWindow.axaml

```
1 | <Window xmlns="https://github.com/avaloniaui"
2 |         xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
3 |         xmlns:vm="using:DjangoToolbox.ViewModels"
4 |         xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
5 |         xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
6 |         mc:Ignorable="d" d:DesignWidth="1100" d:DesignHeight="700"
7 |         x:Class="DjangoToolbox.Views.MainWindow"
8 |         x:DataType="vm:MainWindowViewModel"
9 |         Icon="/Assets/app.ico"
10 |        Title="DjangoToolbox"
11 |        Width="1100" Height="800"
12 |        MinWidth="900" MinHeight="750"
13 |        WindowDecorations="None"
14 |        WindowStartupLocation="CenterScreen"
15 |        Background="{DynamicResource CyberBg}"
16 |        Opacity="0">
17 |
18 |     <Grid RowDefinitions="44,*">
19 |         <!-- ===== Custom Cyberpunk Title Bar ===== -->
20 |         <Border Grid.Row="0"
21 |                 Background="{DynamicResource CyberPanel}"
22 |                 BorderBrush="{DynamicResource CyberYellow}"
23 |                 BorderThickness="0,0,0,1"
24 |                 PointerPressed="OnTitleBarPressed">
25 |             <Grid ColumnDefinitions="Auto,*,Auto">
26 |
27 |                 <TextBlock Grid.Column="0"
28 |                             Text="? DJANGI TOOLBOX"
29 |                             Foreground="{DynamicResource CyberYellow}"
30 |                             FontWeight="Bold"
31 |                             FontSize="14"
32 |                             VerticalAlignment="Center"
33 |                             Margin="16,0,0,0"/>
34 |
35 |                 <StackPanel Grid.Column="2" Orientation="Horizontal">
36 |                     <Button Name="MinimizeBtn" Content="?" Classes="icon" Width="44" Height="32"
37 |                             Background="Transparent" Click="OnMinimize"/>
38 |                     <Button Name="MaximizeBtn" Content="?" Classes="icon" Width="44" Height="32"
39 |                             Background="Transparent" Click="OnMaximizeToggle"/>
40 |                     <Button Name="CloseBtn" Content="?" Classes="icon close" Width="44" Height="32"
41 |                             Background="Transparent"
42 |                             Click="OnClose"/>
43 |                 </StackPanel>
44 |             </Grid>
45 |         </Border>
46 |
47 |         <!-- ===== Content Area ===== -->
48 |         <Grid Grid.Row="1" ColumnDefinitions="220,*">
49 |
50 |             <!-- Nav rail -->
51 |             <Border Grid.Column="0"
52 |                     Background="{DynamicResource CyberPanel}"
53 |                     BorderBrush="{DynamicResource CyberBorder}"
54 |                     BorderThickness="0,0,1,0">
55 |                 <StackPanel Margin="12,20,12,0" Spacing="8">
56 |
57 |                     <TextBlock Text="// MODULES"
58 |                                 Classes="dim"
59 |                                 Margin="8,0,0,8"/>
60 |
61 |                     <Button Command="{Binding NavigateHomeCommand}"
62 |                             HorizontalAlignment="Stretch"
63 |                             HorizontalContentAlignment="Left"
64 |                             Content="? HOME"
65 |                             Classes="glitch"
66 |                             Classes.active="{Binding IsHomeActive}"/>
67 |
68 |                     <Button Command="{Binding NavigateCodeZipperCommand}"
69 |                             HorizontalAlignment="Stretch"
70 |                             HorizontalContentAlignment="Left"
71 |                             Content="? CODEZIPPER"
72 |                             Classes="glitch"
73 |                             Classes.active="{Binding IsZipActive}"/>
74 |
75 |                 </StackPanel>
76 |             </Border>
77 |
78 |             <!-- Content area with page transition -->
```

DJANGI TOOLBOX

// SOURCE CODE EXPORT • 20 file(s) • 2026-10-03 23:51:43

```
80 |         <Border Grid.Column="1" ClipToBounds="True">
81 |             <Grid>
82 |                 <ContentControl Content="{Binding CurrentPage}"/>
83 |
84 |                 <!-- Blocks get generated here at runtime -->
85 |                 <StackPanel Name="DissolveLayer"
86 |                     VerticalAlignment="Top"
87 |                     IsHitTestVisible="False"/>
88 |             </Grid>
89 |         </Border>
90 |
91 |     </Grid>
92 |
93 |     <!-- ===== Boot overlay (covers body only, not title bar) ===== -->
94 |     <Grid Name="BootOverlay"
95 |         Grid.Row="1"
96 |         Background="{DynamicResource CyberBg}"
97 |         IsVisible="{Binding Boot.IsVisible}">
98 |
99 |         <StackPanel VerticalAlignment="Center" HorizontalAlignment="Center"
100 |             Spacing="16" Width="680">
101 |
102 |             <!-- Title with pulse animation -->
103 |             <TextBlock Text="DJANGI TOOLBOX"
104 |                 Classes="hero"
105 |                 FontSize="56"
106 |                 HorizontalAlignment="Center"
107 |                 Foreground="{DynamicResource CyberYellow}">
108 |                 <TextBlock.Effect>
109 |                     <DropShadowEffect Color="#FCE22A" BlurRadius="28"
110 |                         OffsetX="0" OffsetY="0" Opacity="0.9"/>
111 |                 </TextBlock.Effect>
112 |                 <TextBlock.Style>
113 |                     <Style Selector="TextBlock">
114 |                         <Style.Animations>
115 |                             <Animation Duration="0:0:2"
116 |                                 IterationCount="INFINITE"
117 |                                 PlaybackDirection="Alternate">
118 |                                 <KeyFrame Cue="0%">
119 |                                     <Setter Property="Opacity" Value="1"/>
120 |                                 </KeyFrame>
121 |                                 <KeyFrame Cue="100%">
122 |                                     <Setter Property="Opacity" Value="0.65"/>
123 |                                 </KeyFrame>
124 |                             </Animation>
125 |                         </Style.Animations>
126 |                     </Style>
127 |                 </TextBlock.Style>
128 |             </TextBlock>
129 |
130 |             <TextBlock Text="// SYSTEM BOOT SEQUENCE v0.1"
131 |                 Classes="dim"
132 |                 HorizontalAlignment="Center"/>
133 |
134 |             <!-- Boot log panel -->
135 |             <Border Classes="panel accent" Padding="24,20" Margin="0,20,0,0">
136 |                 <StackPanel Spacing="6">
137 |
138 |                     <ItemsControl ItemsSource="{Binding Boot.Lines}" MinHeight="140">
139 |                         <ItemsControl.ItemTemplate>
140 |                             <DataTemplate>
141 |                                 <TextBlock Text="{Binding}"
142 |                                     FontFamily="Consolas"
143 |                                     FontSize="13"
144 |                                     Foreground="{DynamicResource CyberGreen}"/>
145 |                             </DataTemplate>
146 |                         </ItemsControl.ItemTemplate>
147 |                     </ItemsControl>
148 |
149 |                     <!-- Progress bar -->
150 |                     <Grid Margin="0,12,0,0" ColumnDefinitions="Auto,*,Auto">
151 |                         <TextBlock Grid.Column="0"
152 |                             Text="LOAD"
153 |                             Classes="dim"
154 |                             VerticalAlignment="Center"
155 |                             Margin="0,0,12,0"/>
156 |
157 |                         <!-- Track -->
158 |                         <Border Grid.Column="1"
159 |                             Height="6"
160 |                             Background="#1F2A36"
161 |                             VerticalAlignment="Center">
162 |                             <!-- Fill: stretches the full track, then scales on X -->
163 |                             <Border HorizontalAlignment="Stretch"
164 |                                 Background="{DynamicResource CyberYellow}"
165 |                                 RenderTransformOrigin="0,0.5">
166 |                                 <Border.RenderTransform>
167 |                                     <ScaleTransform ScaleX="{Binding Boot.Progress}"/>

```

DJANGI TOOLBOX

// SOURCE CODE EXPORT • 20 file(s) • 2026-10-03 23:51:43

```
168 |                                     </Border.RenderTransform>
169 |                                     <Border.Effect>
170 |                                         <DropShadowEffect Color="#FCE22A"
171 |                                             BlurRadius="12"
172 |                                             OffsetX="0" OffsetY="0"
173 |                                             Opacity="0.85"/>
174 |                                     </Border.Effect>
175 |                                 </Border>
176 |                             </Border>
177 |
178 |                             <TextBlock Grid.Column="2"
179 |                                     Text="{Binding Boot.ProgressPercent}"
180 |                                     Classes="dim"
181 |                                     VerticalAlignment="Center"
182 |                                     Margin="12,0,0,0"/>
183 |                         </Grid>
184 |
185 |                 </StackPanel>
186 |             </Border>
187 |
188 |             <!-- ACCESS GRANTED -->
189 |             <TextBlock Text="ACCESS GRANTED"
190 |                     Classes="hero"
191 |                     FontSize="26"
192 |                     HorizontalAlignment="Center"
193 |                     IsVisible="{Binding Boot.IsComplete}"
194 |                     Foreground="{DynamicResource CyberGreen}"
195 |                     Margin="0,8,0,0">
196 |                 <TextBlock.Effect>
197 |                     <DropShadowEffect Color="#3DFFA0" BlurRadius="24"
198 |                                             OffsetX="0" OffsetY="0" Opacity="0.9"/>
199 |                 </TextBlock.Effect>
200 |             </TextBlock>
201 |
202 |         </StackPanel>
203 |     </Grid>
204 |
205 |     <!-- ===== Resize Grips (invisible, sit on top of edges) ===== -->
206 |     <Grid Name="ResizeGrips" Grid.Row="0" Grid.RowSpan="2">
207 |
208 |         <!-- Edges -->
209 |         <Border Height="6" VerticalAlignment="Top" HorizontalAlignment="Stretch"
210 |                 Background="Transparent"
211 |                 Cursor="SizeNorthSouth"
212 |                 PointerPressed="OnResizeTop"/>
213 |         <Border Height="6" VerticalAlignment="Bottom" HorizontalAlignment="Stretch"
214 |                 Background="Transparent"
215 |                 Cursor="SizeNorthSouth"
216 |                 PointerPressed="OnResizeBottom"/>
217 |         <Border Width="6" HorizontalAlignment="Left" VerticalAlignment="Stretch"
218 |                 Background="Transparent"
219 |                 Cursor="SizeWestEast"
220 |                 PointerPressed="OnResizeLeft"/>
221 |         <Border Width="6" HorizontalAlignment="Right" VerticalAlignment="Stretch"
222 |                 Background="Transparent"
223 |                 Cursor="SizeWestEast"
224 |                 PointerPressed="OnResizeRight"/>
225 |
226 |         <!-- Corners (bigger hit area) -->
227 |         <Border Width="12" Height="12" HorizontalAlignment="Left" VerticalAlignment="Top"
228 |                 Background="Transparent"
229 |                 Cursor="TopLeftCorner"
230 |                 PointerPressed="OnResizeTopLeft"/>
231 |         <Border Width="12" Height="12" HorizontalAlignment="Right" VerticalAlignment="Top"
232 |                 Background="Transparent"
233 |                 Cursor="TopRightCorner"
234 |                 PointerPressed="OnResizeTopRight"/>
235 |         <Border Width="12" Height="12" HorizontalAlignment="Left" VerticalAlignment="Bottom"
236 |                 Background="Transparent"
237 |                 Cursor="BottomLeftCorner"
238 |                 PointerPressed="OnResizeBottomLeft"/>
239 |         <Border Width="12" Height="12" HorizontalAlignment="Right" VerticalAlignment="Bottom"
240 |                 Background="Transparent"
241 |                 Cursor="BottomRightCorner"
242 |                 PointerPressed="OnResizeBottomRight"/>
243 |
244 |     </Grid>
245 | </Grid>
246 | </Window>
```

MainWindow.axaml.cs

F:\Tools\DjangaIToolbox\Views\MainWindow.axaml.cs

```
1 | using System;
2 | using System.Collections.Generic;
3 | using System.Linq;
4 | using System.Threading.Tasks;
5 | using Avalonia;
```

DJANGI TOOLBOX

// SOURCE CODE EXPORT • 20 file(s) • 2026-10-03 23:51:43

```
6 | using Avalonia.Controls;
7 | using Avalonia.Input;
8 | using Avalonia.Interactivity;
9 | using Avalonia.Media;
10 | using Avalonia.VisualTree;
11 | using DjangiToolbox.ViewModels;
12 | using DjangiToolbox.Services;
13 |
14 | namespace DjangiToolbox.Views;
15 |
16 | public partial class MainWindow : Window
17 | {
18 |     // ---- Transition tuning knobs ----
19 |     private const int BlockMinHeight = 35;
20 |     private const int BlockMaxHeight = 85;
21 |     private const int CoverFadeMs = 40; // per-block fade in
22 |     private const int CoverMaxDelayMs = 80; // random stagger during cover
23 |     private const int RevealFadeMs = 60; // per-block fade out
24 |     private const int RevealMaxDelayMs = 80; // random stagger during reveal
25 |     private const double DarkChance = 0.45; // 45% darker block, 55% lighter block
26 |     private const string DarkBlockHex = "#141B24"; // deep slate ? barely lighter than bg
27 |     private const string LightBlockHex = "#1A222C"; // slightly lighter slate
28 |
29 |     private readonly Random _rng = new();
30 |     private MainWindowViewModel? _subscribedVm;
31 |
32 |     public MainWindow()
33 |     {
34 |         InitializeComponent();
35 |     }
36 |
37 |     protected override void OnOpened(EventArgs e)
38 |     {
39 |         base.OnOpened(e);
40 |         _ = RunStartupAsync();
41 |     }
42 |
43 |     protected override void OnDataContextChanged(EventArgs e)
44 |     {
45 |         base.OnDataContextChanged(e);
46 |
47 |         if (_subscribedVm is not null)
48 |         {
49 |             _subscribedVm.BeforePageChange -= PlayCoverAsync;
50 |             _subscribedVm.AfterPageChange -= PlayRevealAsync;
51 |         }
52 |
53 |         if (DataContext is MainWindowViewModel vm)
54 |         {
55 |             vm.BeforePageChange += PlayCoverAsync;
56 |             vm.AfterPageChange += PlayRevealAsync;
57 |             _subscribedVm = vm;
58 |         }
59 |     }
60 |
61 |     // ===== Digital dissolve =====
62 |
63 |     private async Task PlayCoverAsync()
64 |     {
65 |         var layer = this.FindControl<StackPanel>("DissolveLayer");
66 |         if (layer is null) return;
67 |
68 |         layer.Children.Clear();
69 |
70 |         double height = (layer.Parent as Visual)?.Bounds.Height ?? 0;
71 |         if (height <= 0) height = Bounds.Height - 44;
72 |
73 |         // Build blocks whose heights sum to exactly `height`.
74 |         var blocks = new List<Border>();
75 |         double y = 0;
76 |         while (y < height - 1)
77 |         {
78 |             double h = _rng.Next(BlockMinHeight, BlockMaxHeight);
79 |             if (y + h > height) h = height - y;
80 |
81 |             var block = new Border
82 |             {
83 |                 Height = h,
84 |                 Background = new SolidColorBrush(
85 |                     Color.Parse(_rng.NextDouble() < DarkChance ? DarkBlockHex : LightBlockHex)),
86 |                 Opacity = 0
87 |             };
88 |             layer.Children.Add(block);
89 |             blocks.Add(block);
90 |             y += h;
91 |         }
92 |
93 |         var tasks = blocks.Select(b => FadeInBlockAsync(b)).ToList();
```

```
94 |         await Task.WhenAll(tasks);
95 |     }
96 |
97 |     private async Task PlayRevealAsync()
98 |     {
99 |         var layer = this.FindControl<StackPanel>("DissolveLayer");
100 |         if (layer is null) return;
101 |
102 |         var tasks = layer.Children.OfType<Border>()
103 |             .Select(b => FadeOutBlockAsync(b))
104 |             .ToList();
105 |         await Task.WhenAll(tasks);
106 |
107 |         layer.Children.Clear();
108 |     }
109 |
110 |     private async Task FadeInBlockAsync(Border block)
111 |     {
112 |         await Task.Delay(_rng.Next(0, CoverMaxDelayMs));
113 |
114 |         const int steps = 4;
115 |         for (int i = 1; i <= steps; i++)
116 |         {
117 |             block.Opacity = i / (double)steps;
118 |             await Task.Delay(CoverFadeMs / steps);
119 |         }
120 |         block.Opacity = 1;
121 |     }
122 |
123 |     private async Task FadeOutBlockAsync(Border block)
124 |     {
125 |         await Task.Delay(_rng.Next(0, RevealMaxDelayMs));
126 |
127 |         const int steps = 4;
128 |         for (int i = steps - 1; i >= 0; i--)
129 |         {
130 |             block.Opacity = i / (double)steps;
131 |             await Task.Delay(RevealFadeMs / steps);
132 |         }
133 |         block.Opacity = 0;
134 |     }
135 |
136 |     // -----
137 |
138 |     private async Task RunStartupAsync()
139 |     {
140 |         FilePicker.Register(this);
141 |
142 |         // 1) fade window in
143 |         await Task.Delay(80);
144 |         const int steps = 20;
145 |         for (int i = 1; i <= steps; i++)
146 |         {
147 |             Opacity = i / (double)steps;
148 |             await Task.Delay(12);
149 |         }
150 |         Opacity = 1;
151 |         SyncWindowStateUi();
152 |
153 |         // 2) play boot sequence
154 |         if (DataContext is not MainWindowViewModel vm) return;
155 |         await vm.Boot.RunAsync();
156 |
157 |         // hold "ACCESS GRANTED" for a beat
158 |         await Task.Delay(450);
159 |
160 |         // 3) fade out the overlay
161 |         if (this.FindControl<Grid>("BootOverlay") is { } overlay)
162 |         {
163 |             const int fadeSteps = 18;
164 |             for (int i = fadeSteps; i >= 0; i--)
165 |             {
166 |                 overlay.Opacity = i / (double)fadeSteps;
167 |                 await Task.Delay(18);
168 |             }
169 |         }
170 |
171 |         // 4) hide it via the VM (overlay binds IsVisible to Boot.IsVisible)
172 |         vm.Boot.IsVisible = false;
173 |     }
174 |
175 |     protected override void OnPropertyChanged(AvaloniaPropertyChangedEventArgs change)
176 |     {
177 |         base.OnPropertyChanged(change);
178 |
179 |         if (change.Property == WindowStateProperty)
180 |             SyncWindowStateUi();
181 |     }
```

DJANGI TOOLBOX

// SOURCE CODE EXPORT • 20 file(s) • 2026-10-03 23:51:43

```
182 |
183 | private void SyncWindowStateUi()
184 | {
185 |     var isMax = WindowState == WindowState.Maximized;
186 |
187 |     // Change the maximize/restore glyph
188 |     if (this.FindControl<Button>("MaximizeBtn") is { } btn)
189 |         btn.Content = isMax ? "?" : "?";
190 |
191 |     // Hide resize grips when maximized (you can't resize a maximized window)
192 |     if (this.FindControl<Grid>("ResizeGrips") is { } grips)
193 |         grips.IsVisible = !isMax;
194 | }
195 |
196 | private void OnTitleBarPressed(object? sender, PointerPressedEventArgs e)
197 | {
198 |     if (e.GetCurrentPoint(this).Properties.IsLeftButtonPressed)
199 |         BeginMoveDrag(e);
200 | }
201 |
202 | private void OnMinimize(object? sender, RoutedEventArgs e)
203 |     => WindowState = WindowState.Minimized;
204 |
205 | private void OnMaximizeToggle(object? sender, RoutedEventArgs e)
206 |     => WindowState = WindowState == WindowState.Maximized
207 |         ? WindowState.Normal
208 |         : WindowState.Maximized;
209 |
210 | private void OnClose(object? sender, RoutedEventArgs e)
211 |     => Close();
212 |
213 | private void OnResizeTop(object? s, PointerPressedEventArgs e) => BeginResizeDrag(WindowEdge.North, e);
214 | private void OnResizeBottom(object? s, PointerPressedEventArgs e) => BeginResizeDrag(WindowEdge.South, e);
215 | private void OnResizeLeft(object? s, PointerPressedEventArgs e) => BeginResizeDrag(WindowEdge.West, e);
216 | private void OnResizeRight(object? s, PointerPressedEventArgs e) => BeginResizeDrag(WindowEdge.East, e);
217 | private void OnResizeTopLeft(object? s, PointerPressedEventArgs e) => BeginResizeDrag(WindowEdge.NorthWest, e);
218 | private void OnResizeTopRight(object? s, PointerPressedEventArgs e) => BeginResizeDrag(WindowEdge.NorthEast, e);
219 | private void OnResizeBottomLeft(object? s, PointerPressedEventArgs e) => BeginResizeDrag(WindowEdge.SouthWest, e);
220 | private void OnResizeBottomRight(object? s, PointerPressedEventArgs e) => BeginResizeDrag(WindowEdge.SouthEast, e);
221 | }
```

CodeZipperViewModel.cs

F:\Tools\DjangoToolbox\ViewModels\Pages\CodeZipperViewModel.cs

```
1 | using System;
2 | using System.Collections.ObjectModel;
3 | using System.IO;
4 | using System.Linq;
5 | using System.Threading.Tasks;
6 | using Avalonia.Platform.Storage;
7 | using CommunityToolkit.Mvvm.ComponentModel;
8 | using CommunityToolkit.Mvvm.Input;
9 | using DjangoToolbox.Models;
10 | using DjangoToolbox.Services;
11 |
12 | namespace DjangoToolbox.ViewModels.Pages;
13 |
14 | public partial class CodeZipperViewModel : ViewModelBase
15 | {
16 |     public ObservableCollection<FileEntry> Files { get; } = new();
17 |     public ObservableCollection<LogEntry> Logs { get; } = new();
18 |
19 |     [ObservableProperty]
20 |     private string _outputPath = "";
21 |
22 |     public int FileCount => Files.Count;
23 |     public bool HasFiles => Files.Count > 0;
24 |     public string FileCountText => $"{Files.Count} file(s) queued";
25 |
26 |     public CodeZipperViewModel()
27 |     {
28 |         // ObservableCollection raises events, but not PropertyChanged
29 |         // for computed props. So we wire them up manually.
30 |         Files.CollectionChanged += (_, _) =>
31 |         {
32 |             OnPropertyChanged(nameof(FileCount));
33 |             OnPropertyChanged(nameof(HasFiles));
34 |             OnPropertyChanged(nameof(FileCountText));
35 |         };
36 |
37 |         Log("// codezipper online ? add files to begin");
38 |     }
39 |
40 |     private void Log(string message, string level = "INFO")
41 |     {
42 |         Logs.Add(new LogEntry(message, level));
43 |     }
44 | }
```

DJANGI TOOLBOX

// SOURCE CODE EXPORT • 20 file(s) • 2026-10-03 23:51:43

```
45 | // Source extensions we consider "code" for the folder scanner.
46 | private static readonly string[] CodeExtensions =
47 | {
48 |     ".cs", ".xaml", ".axaml", ".json", ".xml", ".config", ".md", ".txt",
49 |     ".yaml", ".yml", ".ps1", ".sh", ".sql", ".html", ".css", ".js", ".ts",
50 |     ".py", ".java", ".go", ".rs", ".cpp", ".h", ".hpp"
51 | };
52 |
53 | private static readonly string[] SkipFolders =
54 | {
55 |     "bin", "obj", ".git", ".vs", ".vscode", "node_modules",
56 |     "packages", "dist", "build", ".idea"
57 | };
58 |
59 | [RelayCommand]
60 | private async Task AddFilesAsync()
61 | {
62 |     var storage = FilePicker.Instance?.StorageProvider;
63 |     if (storage is null) return;
64 |
65 |     var picked = await storage.OpenFilePickerAsync(new FilePickerOpenOptions
66 |     {
67 |         Title = "Select source files",
68 |         AllowMultiple = true
69 |     });
70 |
71 |     if (picked is null || picked.Count == 0) return;
72 |
73 |     int added = 0;
74 |     foreach (var file in picked)
75 |     {
76 |         var path = file.Path.LocalPath;
77 |         if (Files.Any(f => f.Path.Equals(path, StringComparison.OrdinalIgnoreCase)))
78 |             continue;
79 |         Files.Add(new FileEntry(path));
80 |         added++;
81 |     }
82 |
83 |     Log(added > 0
84 |         ? $"// added {added} file(s)"
85 |         : "// no new files (already in queue)");
86 | }
87 |
88 | [RelayCommand]
89 | private async Task AddFolderAsync()
90 | {
91 |     var storage = FilePicker.Instance?.StorageProvider;
92 |     if (storage is null) return;
93 |
94 |     var picked = await storage.OpenFolderPickerAsync(new FolderPickerOpenOptions
95 |     {
96 |         Title = "Select source folder",
97 |         AllowMultiple = false
98 |     });
99 |
100 |     if (picked is null || picked.Count == 0) return;
101 |     var root = picked[0].Path.LocalPath;
102 |
103 |     int added = 0;
104 |     foreach (var file in Directory.EnumerateFiles(root, "*", SearchOption.AllDirectories))
105 |     {
106 |         var segments = file.Split(Path.DirectorySeparatorChar);
107 |         if (segments.Any(s => SkipFolders.Contains(s, StringComparison.OrdinalIgnoreCase)))
108 |             continue;
109 |
110 |         var ext = Path.GetExtension(file);
111 |         if (!CodeExtensions.Contains(ext, StringComparison.OrdinalIgnoreCase))
112 |             continue;
113 |
114 |         if (Files.Any(f => f.Path.Equals(file, StringComparison.OrdinalIgnoreCase)))
115 |             continue;
116 |
117 |         Files.Add(new FileEntry(file));
118 |         added++;
119 |     }
120 |
121 |     Log($"// scanned folder ? added {added} file(s)");
122 | }
123 |
124 | [RelayCommand]
125 | private void RemoveFile(FileEntry? entry)
126 | {
127 |     if (entry is null) return;
128 |     Files.Remove(entry);
129 |     Log("// file removed");
130 | }
131 |
132 | [RelayCommand]
```


DJANGI TOOLBOX

// SOURCE CODE EXPORT • 20 file(s) • 2026-10-03 23:51:43

```
133 | private void ClearAll()
134 | {
135 |     Files.Clear();
136 |     Log("// queue cleared");
137 | }
138 |
139 | [RelayCommand]
140 | private async Task PickOutputAsync()
141 | {
142 |     var storage = FilePicker.Instance?.StorageProvider;
143 |     if (storage is null) return;
144 |
145 |     var file = await storage.SaveFilePickerAsync(new FilePickerSaveOptions
146 |     {
147 |         Title = "Save PDF as",
148 |         SuggestedFileName = "source-code.pdf",
149 |         DefaultExtension = ".pdf",
150 |         FileTypeChoices = new[]
151 |         {
152 |             new FilePickerFileType("PDF document")
153 |             {
154 |                 Patterns = new[] { "*.pdf" }
155 |             }
156 |         }
157 |     });
158 |
159 |     if (file is null) return;
160 |     OutputPath = file.Path.LocalPath;
161 |     Log("// output path set");
162 | }
163 |
164 | [RelayCommand]
165 | private async Task RunAsync()
166 | {
167 |     if (!HasFiles)
168 |     {
169 |         Log("// queue is empty ? add files first", "WARN");
170 |         return;
171 |     }
172 |     if (string.IsNullOrWhiteSpace(OutputPath))
173 |     {
174 |         Log("// no output path ? pick one first", "WARN");
175 |         return;
176 |     }
177 |
178 |     Log("// generating PDF...");
179 |
180 |     try
181 |     {
182 |         var files = Files.ToList();
183 |         var output = OutputPath;
184 |
185 |         await Task.Run(() => PdfGenerator.Generate(output, files));
186 |
187 |         Log($"// PDF written: {System.IO.Path.GetFileName(output)}");
188 |     }
189 |     catch (Exception ex)
190 |     {
191 |         Log($"// error: {ex.Message}", "ERROR");
192 |     }
193 | }
194 | }
```

HomeViewModel.cs

F:\Tools\DjangoToolbox\ViewModels\Pages\HomeViewModel.cs

```
1 | using DjangoToolbox.ViewModels;
2 |
3 | namespace DjangoToolbox.ViewModels.Pages;
4 |
5 | public class HomeViewModel : ViewModelBase
6 | {
7 |     public string Greeting => "DJANGI TOOLBOX";
8 |     public string Subtitle => "// operator console ? v0.1";
9 | }
```

CodeZipperView.axaml

F:\Tools\DjangoToolbox\Views\Pages\CodeZipperView.axaml

```
1 | <UserControl xmlns="https://github.com/avaloniaui"
2 |             xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
3 |             xmlns:vm="using:DjangoToolbox.ViewModels.Pages"
4 |             xmlns:models="using:DjangoToolbox.Models"
5 |             x:Class="DjangoToolbox.Views.Pages.CodeZipperView"
6 |             x:DataType="vm:CodeZipperViewModel">
7 |
8 |     <Grid RowDefinitions="Auto,*,Auto,180" Margin="32,24">
9 |
10 |         <!-- Header -->
```

DJANGI TOOLBOX

// SOURCE CODE EXPORT • 20 file(s) • 2026-10-03 23:51:43

```
11 |         <StackPanel Grid.Row="0" Spacing="4" Margin="0,0,0,20">
12 |             <TextBlock Text="CODEZIPPER" Classes="hero" FontSize="32"/>
13 |             <TextBlock Text="// zip source code into a single PDF"
14 |                 Classes="dim"/>
15 |         </StackPanel>
16 |
17 |         <!-- Sources panel -->
18 |         <Border Grid.Row="1" Classes="panel accent" Margin="0,0,0,16">
19 |             <Grid RowDefinitions="Auto,*,Auto">
20 |
21 |                 <Grid Grid.Row="0" ColumnDefinitions="Auto,Auto,*,Auto"
22 |                     Margin="0,0,0,12">
23 |                     <Button Grid.Column="0"
24 |                         Content="+ ADD FILES"
25 |                         Classes="glitch"
26 |                         Command="{Binding AddFilesCommand}"
27 |                         Margin="0,0,8,0"/>
28 |                     <Button Grid.Column="1"
29 |                         Content="+ ADD FOLDER"
30 |                         Classes="glitch"
31 |                         Command="{Binding AddFolderCommand}"
32 |                         Margin="0,0,8,0"/>
33 |                     <Button Grid.Column="3"
34 |                         Content="CLEAR"
35 |                         Classes="glitch"
36 |                         Command="{Binding ClearAllCommand}"/>
37 |                 </Grid>
38 |
39 |                 <!-- File list -->
40 |                 <Border Grid.Row="1"
41 |                     Background="#0A0E14"
42 |                     BorderBrush="{DynamicResource CyberBorder}"
43 |                     BorderThickness="1"
44 |                     MinHeight="180">
45 |                     <ListBox ItemsSource="{Binding Files}"
46 |                         Background="Transparent"
47 |                         BorderThickness="0"
48 |                         Padding="4">
49 |                         <ListBox.Styles>
50 |                             <Style Selector="ListBoxItem">
51 |                                 <Setter Property="Padding" Value="0"/>
52 |                                 <Setter Property="Margin" Value="0"/>
53 |                                 <Setter Property="Background" Value="Transparent"/>
54 |                             </Style>
55 |                             <Style Selector="ListBoxItem:pointerover /template/ ContentPresenter">
56 |                                 <Setter Property="Background" Value="#131A23"/>
57 |                             </Style>
58 |                             <Style Selector="ListBoxItem:selected /template/ ContentPresenter">
59 |                                 <Setter Property="Background" Value="#18222E"/>
60 |                             </Style>
61 |                         </ListBox.Styles>
62 |
63 |                         <ListBox.ItemTemplate>
64 |                             <DataTemplate DataType="models:FileEntry">
65 |                                 <Grid ColumnDefinitions="*,Auto,Auto" Margin="10,6">
66 |                                     <TextBlock Grid.Column="0"
67 |                                         Text="{Binding Name}"
68 |                                         Foreground="{DynamicResource CyberCyan}"
69 |                                         FontSize="13"
70 |                                         VerticalAlignment="Center"
71 |                                         TextTrimming="CharacterEllipsis"
72 |                                         ToolTip.Tip="{Binding Path}"/>
73 |                                     <TextBlock Grid.Column="1"
74 |                                         Text="{Binding SizeDisplay}"
75 |                                         Classes="dim"
76 |                                         VerticalAlignment="Center"
77 |                                         Margin="12,0"
78 |                                         MinWidth="60"/>
79 |                                     <Button Grid.Column="2"
80 |                                         Content="?"
81 |                                         Classes="icon glitch"
82 |                                         Width="28" Height="24"
83 |                                         Background="Transparent"
84 |                                         Foreground="{DynamicResource CyberRed}"
85 |                                         Command="{Binding $parent[UserControl].
((vm:CodeZipperViewModel)DataContext).RemoveFileCommand}"
86 |                                         CommandParameter="{Binding}"/>
87 |                                 </Grid>
88 |                             </DataTemplate>
89 |                         </ListBox.ItemTemplate>
90 |                     </ListBox>
91 |                 </Border>
92 |
93 |                 <!-- Footer: count -->
94 |                 <TextBlock Grid.Row="2"
95 |                     Text="{Binding FileCountText}"
96 |                     Classes="dim"
97 |                     Margin="4,10,0,0"/>
```

```
98 |
99 |         </Grid>
100 |     </Border>
101 |
102 |     <Grid Grid.Row="2" ColumnDefinitions="*, Auto" Margin="0,0,0,16">
103 |         <!-- Output row -->
104 |         <Border Grid.Column="0" Classes="panel">
105 |             <Grid ColumnDefinitions="Auto,*, Auto">
106 |                 <TextBlock Grid.Column="0"
107 |                     Text="OUTPUT"
108 |                     Foreground="{DynamicResource CyberYellow}"
109 |                     FontWeight="SemiBold"
110 |                     VerticalAlignment="Center"
111 |                     Margin="0,0,16,0"/>
112 |                 <TextBox Grid.Column="1"
113 |                     Text="{Binding OutputPath}"
114 |                     PlaceholderText="no output selected"
115 |                     Background="#0A0E14"
116 |                     Foreground="{DynamicResource CyberCyan}"
117 |                     BorderBrush="{DynamicResource CyberBorder}"
118 |                     BorderThickness="1"
119 |                     Padding="10,8"
120 |                     FontFamily="Consolas"
121 |                     VerticalAlignment="Center"/>
122 |                 <Button Grid.Column="2"
123 |                     Content="BROWSE"
124 |                     Classes="glitch"
125 |                     Command="{Binding PickOutputCommand}"
126 |                     Margin="12,0,0,0"/>
127 |             </Grid>
128 |         </Border>
129 |
130 |         <!-- Action bar -->
131 |         <Border Grid.Column="1" Classes="panel" Margin="12,0,0,0">
132 |             <Button Grid.Column="3"
133 |                 Content="?" RUN"
134 |                 Classes="glitch"
135 |                 Command="{Binding RunCommand}"/>
136 |         </Border>
137 |     </Grid>
138 |
139 |     <!-- ===== LOG PANEL ===== -->
140 |     <Border Grid.Row="4" Classes="panel" Padding="0">
141 |         <Grid RowDefinitions="Auto,*">
142 |
143 |             <Border Grid.Row="0"
144 |                 Background="#0A0E14"
145 |                 BorderBrush="{DynamicResource CyberBorder}"
146 |                 BorderThickness="0,0,0,1"
147 |                 Padding="12,8">
148 |                 <TextBlock Text="// OUTPUT LOG"
149 |                     Foreground="{DynamicResource CyberYellow}"
150 |                     FontSize="12"
151 |                     FontWeight="SemiBold"/>
152 |             </Border>
153 |
154 |             <ScrollView Grid.Row="1"
155 |                 Name="LogScroll"
156 |                 Background="#0A0E14"
157 |                 Padding="12,8"
158 |                 HorizontalScrollBarVisibility="Disabled"
159 |                 VerticalScrollBarVisibility="Auto">
160 |                 <ItemsControl ItemsSource="{Binding Logs}">
161 |                     <ItemsControl.ItemTemplate>
162 |                         <DataTemplate DataType="models:LogEntry">
163 |                             <Border Classes="logRow">
164 |                                 <Grid ColumnDefinitions="Auto,Auto,*" Margin="0,2">
165 |                                     <TextBlock Grid.Column="0"
166 |                                         Text="{Binding TimeText}"
167 |                                         FontFamily="Consolas" FontSize="12"
168 |                                         Foreground="{DynamicResource CyberDim}"
169 |                                         Margin="0,0,10,0"/>
170 |                                     <TextBlock Grid.Column="1"
171 |                                         Text="{Binding Level}"
172 |                                         Classes.info="{Binding IsInfo}"
173 |                                         Classes.warn="{Binding IsWarn}"
174 |                                         Classes.error="{Binding IsError}"
175 |                                         FontFamily="Consolas" FontSize="12"
176 |                                         FontWeight="SemiBold"
177 |                                         MinWidth="52"
178 |                                         Margin="0,0,10,0"/>
179 |                                     <TextBlock Grid.Column="2"
180 |                                         Text="{Binding Message}"
181 |                                         FontFamily="Consolas" FontSize="12"
182 |                                         Foreground="{DynamicResource CyberCyan}"
183 |                                         TextWrapping="Wrap"/>
184 |                                 </Grid>
185 |                             </Border>
```

DJANGI TOOLBOX

// SOURCE CODE EXPORT • 20 file(s) • 2026-10-03 23:51:43

```
186 |                 </DataTemplate>
187 |             </ItemsControl.ItemTemplate>
188 |         </ItemsControl>
189 |     </ScrollView>
190 |
191 |     </Grid>
192 | </Border>
193 |
194 | </Grid>
195 | </UserControl>
```

CodeZipperView.axaml.cs

F:\Tools\DjangiToolbox\Views\Pages\CodeZipperView.axaml.cs

```
1 | using System;
2 | using Avalonia.Controls;
3 | using Avalonia.Threading;
4 | using DjangiToolbox.ViewModels.Pages;
5 |
6 | namespace DjangiToolbox.Views.Pages;
7 |
8 | public partial class CodeZipperView : UserControl
9 | {
10 |     public CodeZipperView()
11 |     {
12 |         InitializeComponent();
13 |     }
14 |
15 |     protected override void OnDataContextChanged(EventArgs e)
16 |     {
17 |         base.OnDataContextChanged(e);
18 |
19 |         if (DataContext is not CodeZipperViewModel vm) return;
20 |
21 |         vm.Logs.CollectionChanged += (_, _) =>
22 |         {
23 |             // Wait for layout pass, then scroll
24 |             Dispatcher.UIThread.Post(
25 |                 () => this.FindControl<ScrollView>("LogScroll")?.ScrollToEnd(),
26 |                 DispatcherPriority.Background);
27 |         };
28 |     }
29 | }
```

HomeView.axaml

F:\Tools\DjangiToolbox\Views\Pages\HomeView.axaml

```
1 | <UserControl xmlns="https://github.com/avaloniaui"
2 |             xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
3 |             xmlns:vm="using:DjangiToolbox.ViewModels.Pages"
4 |             x:Class="DjangiToolbox.Views.Pages.HomeView"
5 |             x:DataType="vm:HomeViewModel">
6 |
7 |     <Grid>
8 |         <StackPanel VerticalAlignment="Center" HorizontalAlignment="Center" Spacing="12">
9 |             <TextBlock Text="{Binding Greeting}" Classes="hero" HorizontalAlignment="Center"/>
10 |            <TextBlock Text="{Binding Subtitle}" Classes="dim" HorizontalAlignment="Center"/>
11 |
12 |            <Border Classes="panel accent" Width="440" Margin="0,24,0,0">
13 |                <StackPanel Spacing="10">
14 |                    <TextBlock Text="SYSTEM STATUS"
15 |                               Foreground="{DynamicResource CyberYellow}"
16 |                               FontWeight="SemiBold"/>
17 |                    <TextBlock Text="All subsystems nominal"/>
18 |                    <TextBlock Text="Choose a tool from the left rail"/>
19 |                </StackPanel>
20 |            </Border>
21 |        </StackPanel>
22 |    </Grid>
23 | </UserControl>
```

HomeView.axaml.cs

F:\Tools\DjangiToolbox\Views\Pages\HomeView.axaml.cs

```
1 | using Avalonia.Controls;
2 |
3 | namespace DjangiToolbox.Views.Pages;
4 |
5 | public partial class HomeView : UserControl
6 | {
7 |     public HomeView()
8 |     {
9 |         InitializeComponent();
10 |     }
11 | }
```